

PROJECT REPORT

ON

LIBRARY MANAGEMENT SYSTEM WITH AI RECOMMENDATION

Submitted in partial fulfillment of the requirements for the degree/program
MCA

Submitted By

Priyanshu Saini
Roll Number: O24MCA110350
Enrollment Number: XYZ
Semester: 4th
Project Type: Individual

Under the Guidance of

Kashish Gupta
[Guide Designation]

[Department Name]
[College / Institute Name]
Chandigarh University

Academic Year: 2025-2026

Bonafide Certificate

This is to certify that the project report entitled **Library Management System with AI Recommendation** submitted by **Priyanshu Saini**, bearing Roll Number **O24MCA110350** and Enrollment Number **XYZ**, is a bonafide record of the project work carried out under my supervision and guidance. This project is submitted in partial fulfillment of the requirements of the **MCA** program during the academic year **2025-2026**.

Project Guide: _____

Head of Department: _____

Institution Seal: _____

Declaration by Student

I hereby declare that the project report entitled **Library Management System with AI Recommendation** is an original work carried out by me under the guidance of **Kashish Gupta**. The project has not been submitted to any other university or institution for the award of any degree or diploma. All sources of information used in this report have been acknowledged in the references section.

Place: _____

Date: 28 May 2026

Signature of Student: _____

Acknowledgement

I would like to express my sincere gratitude to my project guide, faculty members, department, and institution for providing support and guidance throughout the development of this project. I am also thankful to my classmates, friends, and family for their encouragement during the analysis, design, implementation, and testing of the Library Management System.

This project helped me understand full-stack application development, REST API integration, database design, authentication, and systematic testing in a practical software development environment.

Abstract

The **Library Management System with AI Recommendation** is a full-stack web application developed to digitize and simplify the daily operations of an educational library. The objective of the project is to provide students with a convenient platform for searching books, checking availability, requesting books, and tracking issued books, while enabling librarians to manage books, students, issue-return operations, reports, and notifications. The frontend is implemented using React, TypeScript, Vite, and Tailwind CSS. The backend is implemented using Java Spring Boot, Spring Security, JWT authentication, Spring Data JPA, MySQL, and Swagger/OpenAPI. The development approach follows an iterative software development methodology where requirements, design, implementation, testing, and refinement are performed module by module. The final system provides role-based access, secure authentication, structured API communication, persistent database storage, and report generation support. The project demonstrates practical knowledge of frontend-backend integration, database schema design, RESTful services, and testing.

Table of Contents

Submitted By	1
Under the Guidance of	1
Bonafide Certificate	2
Declaration by Student	3
Acknowledgement	4
Abstract	5
Table of Contents	6
List of Figures	9
List of Tables	10
List of Abbreviations	11
Chapter 1: Introduction	12
1.1 Background of the Project	12
1.2 Problem Statement	12
1.3 Objectives of the System	12
1.4 Scope of the Project	12
1.5 Existing System Overview	13
1.6 Proposed System Overview	13
1.7 Technologies Used	13
Chapter 2: Literature Review / System Study	14
2.1 Review of Similar Systems	14
2.2 Comparative Analysis	14
2.3 Development Methodology	14
2.4 Relevant Frameworks and Libraries	14
2.5 Research Gap	15
Chapter 3: System Analysis	16
3.1 Functional Requirements	16
3.2 Non-Functional Requirements	16
3.3 User Requirements	16

3.4 Feasibility Study	16
3.5 System Architecture	17
3.6 Data Flow Diagram	17
Chapter 4: System Design	18
4.1 Use Case Diagram	18
4.2 Entity Relationship Design	18
4.3 Database Schema	19
4.4 API Design	19
4.5 Sequence Flow for Book Issue	19
4.6 Activity Flow for Login	20
4.7 UI/UX Wireframe Overview	20
Chapter 5: System Implementation	21
5.1 Development Environment	21
5.2 Hardware and Software Requirements	21
5.3 Frontend Implementation	21
5.4 Backend Implementation	21
5.5 Module-wise Explanation	21
5.6 Important Code Snippets	22
5.7 Security Features	22
5.8 Version Control	22
Chapter 6: Testing	23
6.1 Testing Strategy	23
6.2 Types of Testing	23
6.3 Test Cases	24
6.4 Bug Reports	25
Chapter 7: Results and Discussion	26
7.1 Output Overview	26
7.2 Screenshots to Add in Final Submission	26
7.3 Performance Evaluation	26
7.4 Comparison with Existing System	26

7.5 Improvements Achieved	27
Chapter 8: Conclusion and Future Enhancements	28
8.1 Conclusion	28
8.2 Key Achievements	28
8.3 Technical Learning Outcomes	28
8.4 Future Scope	28
Chapter 9: References	30
Chapter 10: Appendices	32
Appendix A: Source Code Structure	32
Appendix B: Database Setup	32
Appendix C: Installation Guide	32
Appendix D: User Manual	32
Appendix E: Items to Attach in Final Submission	33

List of Figures

Figure No.	Title
Figure 3.1	High-level system architecture
Figure 3.2	DFD Level 0 for Library Management System
Figure 4.1	Entity relationship overview
Figure 4.2	Use case overview
Figure 4.3	Sequence flow for book issue operation
Figure 4.4	Activity flow for login and dashboard access

List of Tables

Table No.	Title
Table 2.1	Comparative analysis of existing and proposed systems
Table 3.1	Functional requirements
Table 3.2	Non-functional requirements
Table 4.1	Database table structures
Table 4.2	API design summary
Table 6.1	Test case summary

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DFD	Data Flow Diagram
ER	Entity Relationship
JPA	Java Persistence API
JWT	JSON Web Token
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface
UX	User Experience

Chapter 1: Introduction

1.1 Background of the Project

Libraries are an important part of educational institutions because they provide access to books, reference materials, and learning resources. In many institutions, library activities such as maintaining book records, checking availability, issuing books, returning books, and preparing reports are still handled manually or through limited spreadsheet-based systems. Such approaches can lead to delays, data duplication, difficulty in tracking issued books, and lack of real-time visibility.

The Library Management System addresses these challenges by providing a web-based platform where students and librarians can perform their respective tasks digitally. Students can search the book catalog and view their issue history, while librarians can manage inventory, issue and return books, and view operational reports.

1.2 Problem Statement

The existing manual library process requires significant effort to maintain records and often results in slow book search, inaccurate availability information, and difficulty in tracking issued books. Librarians need a reliable system to manage books and students, while students need a simple interface for finding and requesting books. Therefore, a centralized digital solution is required to manage library operations efficiently.

1.3 Objectives of the System

- To provide secure role-based login for students and librarians.
- To maintain a digital catalog of books with availability status.
- To allow students to search, filter, and request books.
- To allow librarians to add, update, delete, issue, and return books.
- To store all user, book, issue, and notification data in MySQL.
- To provide reports for book inventory, issue status, and student activity.
- To document backend APIs using Swagger/OpenAPI.

1.4 Scope of the Project

The scope of the project includes student registration and login, librarian login, book catalog management, issue-return management, notifications, reports, and recommendation support. The system is intended for local deployment in an educational library environment and can be extended for cloud deployment, mobile applications, fine management, and barcode based issue processing in the future.

1.5 Existing System Overview

In a manual system, book records are maintained in registers or spreadsheets. Students must physically ask the librarian about availability, and the librarian must manually check records. Reports are difficult to prepare because data is distributed and not always updated in real time. This makes the process time-consuming and error-prone.

1.6 Proposed System Overview

The proposed system provides a React-based frontend connected to a Java Spring Boot backend. The backend exposes REST APIs and stores data in MySQL using JPA. JWT is used for authentication and authorization. The system provides separate dashboards for students and librarians, improving usability and data accuracy.

1.7 Technologies Used

- React and TypeScript are used to build the frontend interface.
- Vite is used for fast frontend development and build processing.
- Tailwind CSS is used for responsive styling.
- Java Spring Boot is used for backend API development.
- Spring Security and JWT are used for authentication.
- Spring Data JPA is used for database operations.
- MySQL is used as the relational database.
- Swagger/OpenAPI is used for API documentation and testing.

Chapter 2: Literature Review / System Study

2.1 Review of Similar Systems

Library automation systems are commonly used in universities and colleges to handle book cataloging, circulation, user registration, and reporting. Traditional library software often focuses on desktop-based use, while modern systems increasingly use web interfaces, REST APIs, and centralized databases. The present project follows this modern approach by separating the frontend and backend and using API-based communication.

2.2 Comparative Analysis

Feature	Manual System	Proposed System
Book Search	Requires manual checking	Instant search and filtering
Availability	May be outdated	Updated through database records
Issue Records	Stored in registers	Stored in MySQL
Reports	Prepared manually	Generated from backend APIs
Security	No digital authentication	JWT-based authentication
Scalability	Limited	Can be extended with more modules

2.3 Development Methodology

The project follows an iterative development methodology. The system was divided into modules such as authentication, book catalog, issue-return, reports, notifications, and recommendations. Each module was analyzed, implemented, tested, and then integrated with the rest of the application. This approach helped in identifying bugs early and improving the project step by step.

2.4 Relevant Frameworks and Libraries

- React supports component-based UI development and state-driven rendering.
- TypeScript improves code reliability through static typing.
- Spring Boot simplifies backend development and provides production-ready defaults.
- Spring Data JPA reduces boilerplate database access code.
- Spring Security provides authentication and authorization support.
- Swagger provides browser-based API documentation and testing.
- MySQL provides relational data storage with structured table relationships.

2.5 Research Gap

Many simple library systems either use only frontend mock data or provide limited backend integration. Some systems do not provide role-based authentication, API documentation, or structured reports. The proposed system attempts to close this gap by providing a full-stack implementation with React frontend, Spring Boot APIs, JWT security, Swagger documentation, and MySQL persistence.

Chapter 3: System Analysis

3.1 Functional Requirements

Req. ID	Requirement
FR-01	The system shall allow students and librarians to log in securely.
FR-02	The system shall allow new users to register.
FR-03	The system shall display a searchable book catalog.
FR-04	The system shall allow librarians to add, update, and delete books.
FR-05	The system shall allow books to be issued and returned.
FR-06	The system shall maintain student issue history.
FR-07	The system shall show reports and summary statistics.
FR-08	The system shall show notifications to users.
FR-09	The system shall provide recommendation data for students.

3.2 Non-Functional Requirements

Category	Requirement
Security	Protected APIs must require JWT authentication.
Usability	The user interface should be simple and responsive.
Performance	Common pages should load data quickly from backend APIs.
Reliability	Database operations should maintain correct availability counts.
Maintainability	The codebase should be separated into frontend and backend modules.
Portability	The project should run locally using Node.js, Java, Maven, and MySQL.

3.3 User Requirements

- Students require a dashboard to view books, recommendations, and their issued items.
- Librarians require management screens for books, students, issues, returns, and reports.
- Both user roles require secure login and meaningful error messages.
- The institution requires persistent storage of library records.

3.4 Feasibility Study

Technical feasibility: The required tools are open-source or commonly available: React, Java, Spring Boot, MySQL, npm, and Maven. The system can be developed and executed on a normal development machine.

Economic feasibility: The project can be implemented with free development tools and does not require expensive infrastructure for local deployment.

Operational feasibility: The system provides clear role-based screens, so students and librarians can use it with basic computer knowledge.

3.5 System Architecture

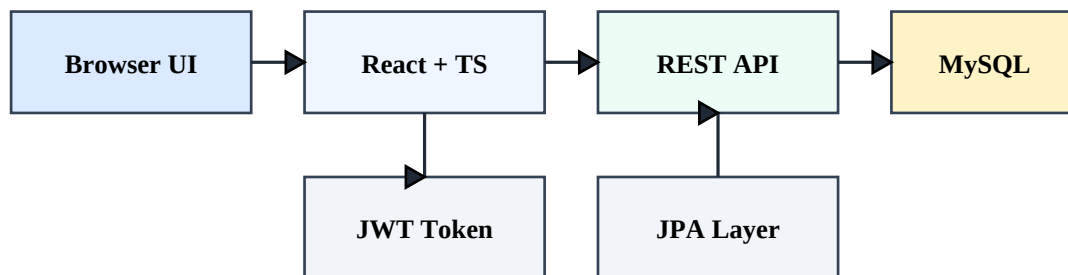


Figure 3.1: High-level system architecture

The architecture separates the user interface, REST API layer, business logic, persistence layer, and database. This separation improves maintainability and allows future extension.

3.6 Data Flow Diagram

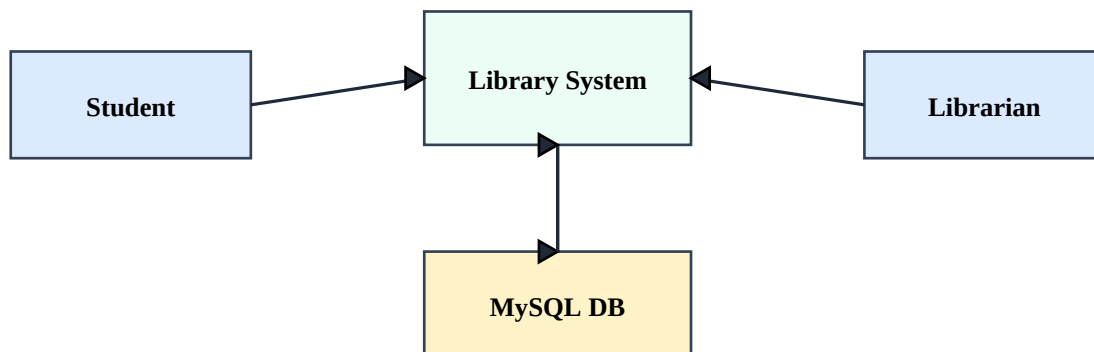


Figure 3.2: DFD Level 0 for Library Management System

The Level 0 DFD shows students and librarians interacting with the central library system, while the backend reads and writes data to MySQL.

Chapter 4: System Design

4.1 Use Case Diagram



Figure 4.2: Use case overview

The use case overview shows the main actions performed by students and librarians. Students mainly search and request books, while librarians manage books, issue/return operations, and reports.

4.2 Entity Relationship Design

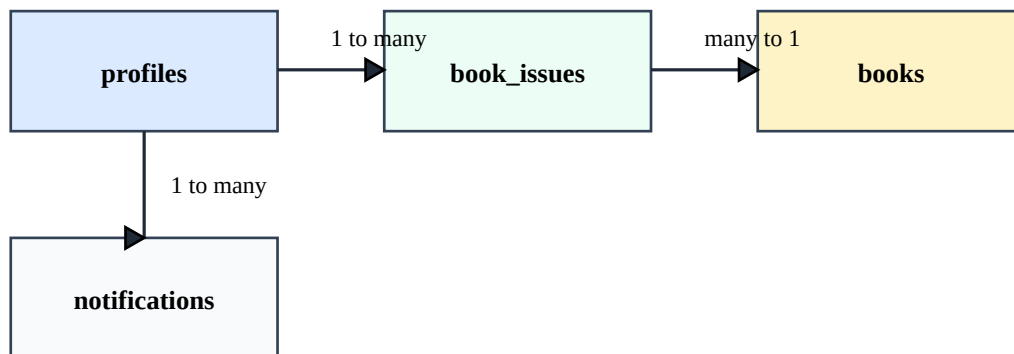


Figure 4.1: Entity relationship overview

The ER overview shows the relationship between users, books, book issue records, and notifications. A user can have multiple issue records and notifications. A book can appear in multiple issue records.

4.3 Database Schema

Table	Important Fields	Purpose
profiles	id, full_name, email, password_hash, role, student_id	Stores user and role information
books	id, title, author, isbn, category, total_copies, available_copies	Stores library book inventory
book_issues	id, book_id, student_id, issued_by, issue_date, due_date, return_date, status	Stores issue and return transactions
notifications	id, profile_id, message, type, is_read, created_at	Stores user notifications

4.4 API Design

Module	Endpoint	Method	Purpose
Auth	/api/auth/register	POST	Register user
Auth	/api/auth/login	POST	Login and receive JWT token
Auth	/api/auth/me	GET	Fetch current user profile
Books	/api/books	GET	Fetch all books
Books	/api/books	POST	Add a new book
Books	/api/books/{id}	PUT	Update book details
Books	/api/books/{id}	DELETE	Delete book
Issues	/api/issues	POST	Issue/request a book
Issues	/api/issues/{id}/return	PUT	Mark book as returned
Reports	/api/reports/summary	GET	Fetch summary report
Recommendations	/api/recommendations	GET	Fetch recommended books

4.5 Sequence Flow for Book Issue

- Student selects an available book from the catalog.
- Frontend sends a POST request to the issue API with book and issue date information.
- Backend validates the logged-in user and book availability.
- Backend creates an issue record and decreases available copies.
- Frontend refreshes the book list and displays the updated availability.

Figure 4.3: Sequence flow for book issue operation

4.6 Activity Flow for Login

- User enters email and password.
- Frontend sends credentials to the login API.
- Backend validates credentials and creates JWT token.
- Frontend stores token in localStorage.
- User is redirected to the dashboard based on role.

Figure 4.4: Activity flow for login and dashboard access

4.7 UI/UX Wireframe Overview

- Login page provides authentication options and demo login support.
- Student dashboard displays books, issue history, and recommendations.
- Book catalog supports search, category filters, and availability filters.
- Librarian dashboard provides management actions for books and issues.
- Reports page presents summary cards, trends, and category statistics.

Chapter 5: System Implementation

5.1 Development Environment

Component	Technology
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	Java 17, Spring Boot
Database	MySQL
Authentication	JWT with Spring Security
API Documentation	Swagger/OpenAPI
Build Tools	npm and Maven

5.2 Hardware and Software Requirements

- Processor: Intel i3 or above recommended.
- RAM: Minimum 4 GB, 8 GB recommended.
- Operating System: Windows 10/11 or compatible OS.
- Software: Node.js, Java 17, Maven, MySQL, VS Code or IntelliJ IDEA.
- Browser: Chrome, Edge, or Firefox.

5.3 Frontend Implementation

The frontend is implemented using React components. The pages folder contains main screens such as StudentDashboard, BookCatalog, LibrarianDashboard, IssueReturnPage, ReportsPage, and AIRecommendations. Reusable UI elements such as Button, Badge, Input, and Modal are placed in the components folder.

5.4 Backend Implementation

The backend follows a layered Spring Boot architecture. Controllers expose REST APIs, services contain business logic, repositories interact with the database, and model classes define database entities. Security classes handle JWT authentication and request filtering.

5.5 Module-wise Explanation

- **Authentication Module:** Handles registration, login, token generation, and current user profile.
- **Book Module:** Handles book listing, add, update, delete, and trending book APIs.
- **Issue Module:** Handles book issue requests, return processing, and issue history.
- **Student Module:** Allows librarians to view student records.

- **Reports Module:** Generates summary statistics and category-based data.
- **Notification Module:** Displays and updates user notifications.
- **Recommendation Module:** Suggests books using rule-based backend logic.

5.6 Important Code Snippets

Frontend API helper:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ??
'http://localhost:8080/api';

export async function apiFetch<T>(path: string, options: RequestInit = {}):
Promise<T> {
  const token = localStorage.getItem('library_token');
  const res = await fetch(`${API_BASE_URL}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
  });
  return res.json() as Promise<T>;
}
```

Database connection configuration:

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/library_management?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=your_mysql_password
spring.jpa.hibernate.ddl-auto=update
springdoc.swagger-ui.path=/swagger-ui.html
```

5.7 Security Features

- JWT token is used to authenticate protected API requests.
- Passwords are stored as hashed values in the backend.
- Role-based access separates student and librarian operations.
- Unauthorized requests return authentication errors.
- CORS configuration allows frontend requests from local development URLs.

5.8 Version Control

The project can be maintained using Git version control. Git helps track source code changes, manage versions, and collaborate during future development.

Chapter 6: Testing

6.1 Testing Strategy

Testing was performed at multiple levels. Frontend type checking was used to detect TypeScript errors. Backend Maven build was used to verify Java compilation. API testing was performed using Swagger and direct HTTP requests. Integration testing was performed by logging in as student and librarian and verifying API responses.

6.2 Types of Testing

- **Unit Testing:** Individual service and utility logic can be tested separately.
- **Integration Testing:** Frontend and backend APIs were checked together.
- **System Testing:** Login, book catalog, reports, issue history, and recommendations were verified.
- **Validation Testing:** Invalid login and unauthorized access cases were checked.

6.3 Test Cases

Test Case ID	Input	Expected Output	Actual Output	Status
TC-01	Valid librarian email and password	Login successful with token	Login successful	Pass
TC-02	Valid student email and password	Login successful with token	Login successful	Pass
TC-03	Invalid password	Error message	Error returned	Pass
TC-04	Open Swagger URL	API documentation visible	Swagger responded	Pass
TC-05	Fetch /api/books	List of books	Seeded books returned	Pass
TC-06	Fetch /api/students as librarian	Student list	Student list returned	Pass
TC-07	Fetch /api/reports/summary	Summary statistics	Report data returned	Pass
TC-08	Fetch /api/issues/my as student	Student issue history	Issue history returned	Pass
TC-09	Fetch /api/recommendations	Recommended books	Recommendations returned	Pass
TC-10	Frontend typecheck	No TypeScript errors	Typecheck passed	Pass
TC-11	Frontend production build	Build successful	Build passed	Pass
TC-12	Backend Maven test/build	Build success	Build success	Pass

6.4 Bug Reports

Bug	Cause	Fix
Missing shared TypeScript types	The project referenced common interfaces but lacked a central type file.	Added src/types.ts with Profile, Book, BookIssue, Notification, and role definitions.
Frontend profile reference error	BookCatalog used profile without reading it from AuthContext.	Imported useAuth and read profile inside BookCatalog.
Supabase dependency not required	Original frontend used Supabase-style structure.	Replaced it with Java API helper and MySQL-backed Spring Boot APIs.
Database persistence requirement	Frontend-only/mock data was not enough for real storage.	Added Java Spring Boot backend connected to MySQL.

Chapter 7: Results and Discussion

7.1 Output Overview

The system provides separate user experiences for students and librarians. After login, the user is redirected to the correct dashboard based on role. Students can access the catalog, issue history, and recommendations. Librarians can manage books, issue/return records, student records, and reports.

7.2 Screenshots to Add in Final Submission

- Login page screenshot.
- Student dashboard screenshot.
- Book catalog screenshot.
- Book details modal screenshot.
- Librarian dashboard screenshot.
- Issue/return page screenshot.
- Reports page screenshot.
- Swagger UI screenshot.
- MySQL database tables screenshot.

7.3 Performance Evaluation

The application is suitable for local development and small to medium educational library use cases. The frontend is lightweight because it is built using Vite and React. The backend uses Spring Boot and JPA, which are reliable for structured business applications. MySQL provides stable relational storage for users, books, issue records, and notifications.

7.4 Comparison with Existing System

Aspect	Existing Manual System	Developed System
Search	Manual and slow	Fast search and filters
Records	Registers/spreadsheets	MySQL database
Authentication	Not available	JWT login
Reports	Manual calculation	API-based report summary
Access	Only through librarian	Student and librarian dashboards
Accuracy	Prone to human errors	Centralized data updates

7.5 Improvements Achieved

- Library data is stored in a structured relational database.
- Frontend is connected to backend APIs instead of direct mock data.
- Role-based authentication improves security and access control.
- Swagger makes backend testing easier.
- Reports provide better visibility into library operations.
- The system can be extended for deployment, mobile apps, and advanced analytics.

Chapter 8: Conclusion and Future Enhancements

8.1 Conclusion

The Library Management System successfully demonstrates a full-stack approach to managing library operations. The project includes a React frontend, Java Spring Boot backend, MySQL database, JWT authentication, Swagger API documentation, and role-based functionality for students and librarians. The major objectives of book catalog management, issue-return handling, reporting, and user authentication have been achieved.

The project helped in understanding software development lifecycle activities such as requirement analysis, database design, API development, frontend-backend integration, testing, and documentation. It also provides a strong foundation for future improvement and deployment.

8.2 Key Achievements

- Implemented role-based student and librarian workflows.
- Connected frontend to Spring Boot REST APIs.
- Connected backend to MySQL database.
- Added JWT-based authentication.
- Added Swagger API documentation.
- Prepared report and testing structure according to academic guidelines.

8.3 Technical Learning Outcomes

- Understanding of React component-based frontend development.
- Understanding of REST API integration.
- Understanding of Spring Boot layered architecture.
- Understanding of JPA entity relationships and repositories.
- Understanding of JWT authentication and role-based access.
- Understanding of MySQL database schema design.

8.4 Future Scope

- Fine calculation for overdue books.
- Email or SMS notifications.
- Barcode or QR-based issue and return system.
- Book reservation and waitlist system.
- PDF and Excel report export.

- Cloud deployment with production database configuration.
- Mobile application support.
- Real AI recommendation model integration.

Chapter 9: References

1. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
2. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
3. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts*. McGraw-Hill.
4. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson.
5. Oracle. (2024). *Java Platform, Standard Edition Documentation*. <https://docs.oracle.com/en/java/>
6. Spring. (2024). *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/>
7. Spring. (2024). *Spring Security Reference Documentation*. <https://docs.spring.io/spring-security/>
8. Spring. (2024). *Spring Data JPA Reference Documentation*. <https://docs.spring.io/spring-data/jpa/>
9. Hibernate. (2024). *Hibernate ORM User Guide*. <https://hibernate.org/orm/documentation/>
10. MySQL. (2024). *MySQL Reference Manual*. <https://dev.mysql.com/doc/>
11. React. (2024). *React Documentation*. <https://react.dev/>
12. Microsoft. (2024). *TypeScript Handbook*. <https://www.typescriptlang.org/docs/>
13. Vite. (2024). *Vite Documentation*. <https://vitejs.dev/guide/>
14. Tailwind Labs. (2024). *Tailwind CSS Documentation*. <https://tailwindcss.com/docs>
15. OpenAPI Initiative. (2024). *OpenAPI Specification*. <https://spec.openapis.org/oas/latest.html>
16. Swagger. (2024). *Swagger UI Documentation*. <https://swagger.io/tools/swagger-ui/>
17. IETF. (2015). *JSON Web Token (JWT), RFC 7519*. <https://www.rfc-editor.org/rfc/rfc7519>
18. OWASP Foundation. (2021). *OWASP Top 10 Web Application Security Risks*. <https://owasp.org/www-project-top-ten/>

19. MDN Web Docs. (2024). *Fetch API*.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
20. Apache Maven Project. (2024). *Apache Maven Documentation*.
<https://maven.apache.org/guides/>
21. Lucide. (2024). *Lucide Icons Documentation*. <https://lucide.dev/>

Chapter 10: Appendices

Appendix A: Source Code Structure

```
Book-manegement-main/  
src/  
components/  
context/  
data/  
lib/  
pages/  
types.ts  
backend/  
src/main/java/com/library/  
config/  
controller/  
dto/  
exception/  
mapper/  
model/  
repository/  
security/  
service/  
src/main/resources/application.properties  
public/  
package.json  
vite.config.ts
```

Appendix B: Database Setup

```
CREATE DATABASE library_management;  
  
-- Backend can also create the database automatically when  
-- createDatabaseIfNotExist=true is present in the JDBC URL.
```

Appendix C: Installation Guide

- Install Java 17, Node.js, Maven, and MySQL.
- Update backend/src/main/resources/application.properties with MySQL username and password.
- Run backend using Maven: mvn spring-boot:run.
- Run frontend using npm run dev.
- Open frontend at <http://localhost:5173>.
- Open Swagger at <http://localhost:8080/swagger-ui.html>.

Appendix D: User Manual

- Student logs in and opens the dashboard.
- Student searches books from the catalog.
- Student requests an available book.

- Librarian logs in and opens librarian dashboard.
- Librarian adds, edits, deletes, issues, and returns books.
- Librarian checks reports and student records.

Appendix E: Items to Attach in Final Submission

- Complete source code printout or repository reference, as required by the institute.
- Database dump file.
- Final screenshots of working application screens.
- Signed bonafide certificate.
- Guide approval signature.